

Prelab

Before beginning Lab 1, you will set up a coding workspace containing:

- A folder/github repo for your work
- A python environment containing the required packages
- Jupyter Notebook support
- An AI coding agent

Some definitions to help you:

- A project folder will keep all the data, code, notebooks, and figures for one project together.
- A Github Repository is a project folder whose changes can be tracked and backed up using Git. If you want to save your progress and reference your work in the future, this is recommended. Otherwise, I'm only requiring a notebook to be submitted with inline figures, so it's up to you whether or not to create a repo.
- A Python environment is an isolated installation of Python and packages that you want to use specifically for some project(s). It can be used as an organizational tool.
- An AI coding agent (I'm sure you know what this is already) can help you by explaining, writing, debugging, and revising code. Its output may be incorrect, so you remain responsible for checking any code you use.
- "The Jupyter Notebook is a web-based interactive computing platform. The notebook combines live code, equations, narrative text, visualizations, interactive dashboards and other media." (re: Project Jupyter)

1. Install Miniconda

Before anything else, install Miniconda and follow the Miniconda installation instructions for your operating system. This gives you Python, Conda, and pip, and it's needed no matter which program you use to run notebooks, so start here regardless of what you pick below.

Mac:

```
curl -O https://repo.anaconda.com/miniconda/Miniconda3-latest-MacOSX-arm64.sh  
bash Miniconda3-latest-MacOSX-arm64.sh
```

(Intel Macs: replace arm64 with x86_64)

Windows:

```
curl https://repo.anaconda.com/miniconda/Miniconda3-latest-Windows-x86_64.exe --output  
.\Miniconda3-latest-Windows-x86_64.exe  
start .\Miniconda3-latest-Windows-x86_64.exe
```

This opens the installer. Click through it, and leave "Add Miniconda3 to PATH" unchecked (the default). You'll use the Anaconda Prompt from the Start menu instead of a regular terminal.

Once installed, close and reopen your terminal and confirm it worked:

conda --version

2. Figure out how you will run notebooks

For the labs, you will complete your work in a Jupyter notebook. You may use whichever notebook-compatible program you prefer:

- [JupyterLab](#): A browser-based interface designed specifically for notebooks
 - If you choose this, wait to install it until step 3. You'll install it directly into your Conda environment.
- [Visual Studio Code](#): Download and install it, then install [Python extension](#) and [Jupyter extension](#)
- [Cursor](#): Similar to VS Code, with a built-in AI coding agent. Download and install it, then install Python and Jupyter (from Microsoft) in the Extensions panel

Your submitted work must be an .ipynb Jupyter notebook + a pdf of the .ipynb file with inline figures displayed, which can be created here, or with nbconvert (although I find this has trouble with inline figures).

3. Create a project folder and Python environment

Open a terminal (Terminal on Mac, Anaconda Prompt on Windows). Navigate to wherever you want your Lab 1 folder to live, for example your Desktop or Documents:

cd Desktop

Then create the folder:

mkdir lab-1 && cd lab-1
mkdir data figures notebooks

You may also create a GitHub repository for the project by following the GitHub repository quickstart.

Now create your environment:

conda create -n lab1 python=3.12
conda activate lab1

If you're using JupyterLab, install it into this environment now:

conda install jupyterlab ipykernel

Then launch it from inside the lab-1 folder:

jupyter lab

This opens JupyterLab in your browser, with lab-1 already showing in the file browser on the left.

If you're using VS Code or Cursor, install ipykernel for now:

```
conda install ipykernel
```

Then open the app (which you already installed in step 2) and use File > Open Folder to open lab-1.

4. Install an integrated AI coding agent

Next, choose an AI coding agent that works inside the program you selected above by being active in the program terminal. We can call this a command-line interface (CLI).

I recommend that your AI agent is integrated into your coding program so that it can help you create, explain, edit, and debug code while you work. Opening ChatGPT in another window while you're coding is inefficient and can result in a lot of errors.

You can choose something like Claude Code, Codex, or Gemini. You will run it from the terminal inside your project folder, where it can inspect, explain, and edit your project files.

**Free option: Gemini CLI gives you 1,000 requests/day with just a Google account. Codex CLI is also free with a free-tier ChatGPT account, but its limits are lower (specific limit values are not public). Claude Code has no free tier, so for this you need Pro (\$20/mo) or API credits.*

The simplest way to install any of these agents is through npm, which is included with Node.js. First, [download and install Node.js](#). Open the terminal inside your chosen program and confirm the installation with `node --version` and `npm --version`.

Then install your agent of choice:

```
npm install -g @anthropic-ai/claude-code  
npm install -g @openai/codex  
npm install -g @google/gemini-cli
```

Follow whatever sign-in prompts are given for each agent.

5. Use the agent to finish setting up your environment

Now launch your AI agent from the terminal (use `claude`, `codex`, or `gemini` in the terminal inside Jupyterlab, VS Code, etc) while your Lab 1 folder is open and lab1 is activated. Ask it to install the packages that support Jupyter notebooks, loading single-cell and spatial count data, running PCA, running NMF, running GLM-PCA, and creating visualizations. Make sure that all packages are installed inside the lab1 environment, not globally, and ask it to list what it plans to install before installing anything. Explicitly require the agent to confirm the active environment before installing

After installing the packages, save a record of the environment:

```
conda env export -n lab1 --no-builds > environment.yml
```

(If you used venv and pip instead of Conda, run `python -m pip freeze > requirements.txt`)

6. Connect the env to a Jupyter notebook

Create a prelab .ipynb inside your Lab 1 folder and select your new environment as its kernel. In VS Code or Cursor, click Select Kernel in the upper-right corner of the notebook. In JupyterLab, click Kernel and then select Change Kernel. The kernel is the Python process that runs your notebook. When you select your environment as the kernel, the notebook is given access to the packages installed in that environment.

From within the notebook, verify it's actually using the right environment:

```
import sys
print(sys.executable)
```

This path should point inside your lab-1 environment.

Now ask your agent in the terminal to make a code block in the notebook that lists out all the packages and their versions that you installed in your environment. You can also experiment further with the agent's capabilities.

If you have any questions, please come to OH or email me at sdharma5@jhu.edu!